

Week 9: Mini-Project: System Diagnostics Reporter

Goal

Build a small command-line tool that reports useful information about whichever machine it runs on (yours or anyone else's), and ship it as a public GitHub repo on your [dev-portfolio](#) profile.

By the end you'll have:

- A Node.js CLI that collects system info and writes a clean report
- A shell script that runs the CLI twice (once successfully, once with a failure) and separates the streams into two log files
- A [.gitignore](#) that keeps logs and [node_modules/](#) out of the repo
- A [README.md](#) explaining how to run the tool on Windows and macOS
- A repo on GitHub linked from your [dev-portfolio](#)

This project is your first piece of work as a **systems-aware** developer. You're not just writing a webpage. You're writing something that interacts with the OS, handles failure cleanly, and ships with documentation. That's the bar for the rest of the bootcamp.

Requirements Checklist

- Local repo initialised: [system-diagnostics/](#)
 - [package.json](#) with at least a [start](#) and a [report](#) script
 - [diagnostics.js](#) reads system metrics via Node's built-in [os](#) module
 - [diagnostics.js](#) accepts a `--label <name>` argument and throws cleanly when it's missing
 - [report.sh](#) (or [report.cmd](#) for Windows-without-WSL) runs the CLI and writes:
 - [report.out.log](#) containing the successful output
 - [report.err.log](#) containing the error from the failing run
 - [.gitignore](#) excludes [node_modules/](#), [*.log](#), [.env](#), [.DS_Store](#)
 - [README.md](#) documents how to run the project on both Windows (WSL2 or Git Bash) and macOS
 - Repo pushed to GitHub
 - Link to the repo added to your [dev-portfolio](#) README under "Bootcamp Projects"
-

Setup

```
# Create and enter the project folder
mkdir system-diagnostics
cd system-diagnostics

# Initialise as a Node project
npm init -y

# Create the source files
touch diagnostics.js report.sh .gitignore README.md
chmod +x report.sh

# Initialise Git
git init
```

Step 1: diagnostics.js

This script collects system info, requires a `--label` argument, and prints a formatted report. If the label is missing, it writes a clear error to stderr and exits with status 1.

```
// diagnostics.js
// A lightweight Node CLI that reports system information.

const os = require("os");

function parseArgs(argv) {
  const args = argv.slice(2);
  const labelIndex = args.indexOf("--label");
  if (labelIndex === -1 || !args[labelIndex + 1]) {
    return { label: null };
  }
  return { label: args[labelIndex + 1] };
}

function gather() {
  const totalGB = (os.totalmem() / 1024 ** 3).toFixed(2);
  const freeGB = (os.freemem() / 1024 ** 3).toFixed(2);
  const uptimeHours = (os.uptime() / 3600).toFixed(2);

  return {
    hostname: os.hostname(),
    platform: os.platform(),
    arch: os.arch(),
    cpuCount: os.cpus().length,
    totalMemoryGB: totalGB,
    freeMemoryGB: freeGB,
    uptimeHours,
    nodeVersion: process.version,
  };
}

function main() {
  const { label } = parseArgs(process.argv);

  if (!label) {
    console.error("ERR_MISSING_LABEL: pass --label <name>, e.g. --label morning-check");
    process.exit(1);
  }

  const data = gather();
  console.log("=== System Diagnostics Report ===");
  console.log(`Label:          ${label}`);
  console.log(`Generated:       ${new Date().toISOString()}`);
  console.log(`Hostname:        ${data.hostname}`);
  console.log(`Platform:        ${data.platform} (${data.arch})`);
  console.log(`CPU cores:       ${data.cpuCount}`);
  console.log(`Memory:          ${data.freeMemoryGB} GB free / ${data.totalMemoryGB} GB
total`);
  console.log(`Uptime:          ${data.uptimeHours} hours`);
}
```

```
console.log(`Node runtime: ${data.nodeVersion}`);  
console.log("=====");  
}  
  
main();
```

Step 2: Add scripts to package.json

Open `package.json` in VS Code and update the `scripts` section:

```
{  
  "scripts": {  
    "start": "node diagnostics.js --label default",  
    "report": "node diagnostics.js"  
  }  
}
```

Test it:

```
npm start  
# Should print the full report with label "default"  
  
npm run report -- --label morning-check  
# Same report, but with your chosen label
```

The `--` between `npm run report` and `--label morning-check` is npm's way of saying "pass everything after this to the underlying command, not to npm itself."

Step 3: report.sh (macOS, Linux, WSL2, Git Bash)

A shell script that runs the diagnostic twice, capturing stdout and stderr into separate files.

```
#!/usr/bin/env bash
# report.sh: run diagnostics twice and separate the streams

echo "======"
echo "Starting diagnostics run"
echo "======"

# Step 1: log basic context
{
echo "Run started at: $(date)"
echo "Working directory: $(pwd)"
echo "Node version: $(node --version)"
echo ""
} > report.out.log

# Step 2: successful run, stdout appended to report.out.log
node diagnostics.js --label scheduled-check >> report.out.log 2>> report.err.log

# Step 3: failing run (no --label argument), stderr appended to report.err.log
echo "" >> report.err.log
echo "Run at: $(date)" >> report.err.log
node diagnostics.js >> report.out.log 2>> report.err.log

echo "Done."
echo "Inspect report.out.log (success output) and report.err.log (errors)."
```

Run it:

```
chmod +x report.sh
./report.sh
```

Windows fallback: report.cmd

If you're on Windows without WSL2 (just Git Bash), the script above still works inside Git Bash. If you want a pure PowerShell/cmd version for whatever reason, create `report.cmd`:

```
@echo off
echo =====
echo Starting diagnostics run
echo =====

(
echo Run started at: %DATE% %TIME%
echo Working directory: %CD%
for /f %i in ('node --version') do echo Node version: %i
echo.
) > report.out.log

node diagnostics.js --label scheduled-check 1>> report.out.log 2>> report.err.log

echo. >> report.err.log
echo Run at: %DATE% %TIME% >> report.err.log
node diagnostics.js 1>> report.out.log 2>> report.err.log

echo Done.
echo Inspect report.out.log and report.err.log.
echo =====
```

Step 4: .gitignore

```
# Dependencies
node_modules/

# Generated logs (do not commit run artefacts)
*.log

# Environment files
.env

# macOS / Windows / editor noise
.DS_Store
Thumbs.db
.vscode/
```

Step 5: README.md

```
# system-diagnostics
```

A tiny Node.js CLI that reports system information. Built as a Week 9 mini-project for the UpliftHub Bootcamp.

```
## What it does
```

Collects hostname, platform, CPU count, memory usage, uptime, and Node version, then prints a labelled report. Errors cleanly when invoked without the required `--label` argument, writing the failure to stderr.

```
## Run it
```

```
### macOS / Linux / WSL2 / Git Bash
```

```
```bash
npm install
node diagnostics.js --label morning-check
```

Or run the bundled report shell script:

```
./report.sh
```

## Windows (PowerShell or cmd)

```
npm install
node diagnostics.js --label morning-check
report.cmd
```

## Output

- `report.out.log` contains the normal output of every run.
- `report.err.log` contains the diagnostics from any failed run (e.g. forgetting to pass `--label`).

## Why this exists

Practising the stdout / stderr separation pattern that every well-behaved CLI tool uses, plus reading system info from Node's built-in `os` module.

```

Step 6: Commit and push to GitHub

```bash
# Stage everything (respecting .gitignore)
git add .

# First commit
git commit -m "feat: initial system diagnostics CLI with report script"

# Create an empty repo on GitHub called "system-diagnostics", then:
git remote add origin https://github.com/<your-username>/system-diagnostics.git
git branch -M main
git push -u origin main
```

Step 7: Link it from your dev-portfolio

In your `dev-portfolio` repo (the one you built in Week 8), edit `README.md` and add an entry under “Bootcamp Projects”:

```
- system-diagnostics: A Node CLI that reports system info, with proper stdout/stderr separation. [Repo] (https://github.com/<your-username>/system-diagnostics)
```

Commit, push, and you're done.

Stretch Goals (Optional)

- Add a `--json` flag that outputs the report as JSON instead of human-readable text.
- Add a `--quiet` flag that suppresses everything except errors. Useful when scripting.
- Add a second exit code: 1 for “missing argument”, 2 for “system info collection failed”. Document them in the README.
- Write a `package.json` “engines” field declaring which Node versions are supported.
- Add a GitHub Actions workflow that runs `node diagnostics.js --label ci` on every push, so the project's CI sanity-checks itself.

Expected Output

Running `./report.sh` from a fresh checkout should produce something like:

Terminal:

```
=====
Starting diagnostics run
=====
Done.
Inspect report.out.log (success output) and report.err.log (errors).
=====
```

report.out.log:

```
Run started at: Mon May 19 14:22:01 WAT 2026
Working directory: /home/sani/projects/system-diagnostics
Node version: v22.11.0

=== System Diagnostics Report ===
Label:          scheduled-check
Generated:      2026-05-19T13:22:02.451Z
Hostname:      sani-laptop
Platform:      linux (x64)
CPU cores:     8
Memory:        4.21 GB free / 15.50 GB total
Uptime:        3.45 hours
Node runtime:  v22.11.0
=====
```

report.err.log:

```
Run at: Mon May 19 14:22:02 WAT 2026
ERR_MISSING_LABEL: pass --label <name>, e.g. --label morning-check
```

Submission

- Repo URL: <https://github.com/<your-username>/system-diagnostics>
- Confirm the repo link is added to [dev-portfolio/README.md](#)
- Drop both links in the Week 9 thread in the WhatsApp group by Friday